

```
In [4]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [5]: df = pd.read_csv(r'C:\Users\Sowriatwork\Desktop\Customer Churn.csv')
df
df.head()
```

Out[5]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	..
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	..
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	..
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	..
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	..
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	..

5 rows × 21 columns



```
In [7]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerID            7043 non-null   object
1   gender                 7043 non-null   object
2   SeniorCitizen          7043 non-null   int64
3   Partner                7043 non-null   object
4   Dependents             7043 non-null   object
5   tenure                 7043 non-null   int64
6   PhoneService           7043 non-null   object
7   MultipleLines          7043 non-null   object
8   InternetService        7043 non-null   object
9   OnlineSecurity         7043 non-null   object
10  OnlineBackup           7043 non-null   object
11  DeviceProtection       7043 non-null   object
12  TechSupport            7043 non-null   object
13  StreamingTV            7043 non-null   object
14  StreamingMovies        7043 non-null   object
15  Contract               7043 non-null   object
16  PaperlessBilling       7043 non-null   object
17  PaymentMethod          7043 non-null   object
18  MonthlyCharges         7043 non-null   float64
19  TotalCharges           7043 non-null   object
20  Churn                  7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

```
In [8]: df["TotalCharges"] = df["TotalCharges"].replace(" ", "0")
df["TotalCharges"] = df["TotalCharges"].astype("float")
```

```
In [9]: df.info
```

```
Out[9]: <bound method DataFrame.info of
0      7590-VHVEG  Female      0      Yes      No      1
1      5575-GNVDE   Male      0      No      No     34
2      3668-QPYBK   Male      0      No      No      2
3      7795-CFOWC   Male      0      No      No     45
4      9237-HQITU   Female     0      No      No      2
...
7038  6840-RESVB    Male      0      Yes     Yes     24
7039  2234-XADUH    Female     0      Yes     Yes     72
7040  4801-JZAZL    Female     0      Yes     Yes     11
7041  8361-LTMKD    Male      1      Yes     No      4
7042  3186-AJIEK    Male      0      No      No     66
```

```

      PhoneService      MultipleLines  InternetService  OnlineSecurity  ...  \
0           No      No phone service              DSL              No  ...
1           Yes              No              DSL              Yes  ...
2           Yes              No              DSL              Yes  ...
3           No      No phone service              DSL              Yes  ...
4           Yes              No      Fiber optic              No  ...
...
7038        Yes              Yes              DSL              Yes  ...
7039        Yes              Yes      Fiber optic              No  ...
7040          No      No phone service              DSL              Yes  ...
7041        Yes              Yes      Fiber optic              No  ...
7042        Yes              No      Fiber optic              Yes  ...
```

```

      DeviceProtection  TechSupport  StreamingTV  StreamingMovies      Contract  \
0           No           No           No           No      Month-to-month
1           Yes           No           No           No           One year
2           No           No           No           No      Month-to-month
3           Yes           Yes           No           No           One year
4           No           No           No           No      Month-to-month
...
7038        Yes           Yes           Yes           Yes           One year
7039        Yes           No           Yes           Yes           One year
7040          No           No           No           No      Month-to-month
7041          No           No           No           No      Month-to-month
7042        Yes           Yes           Yes           Yes           Two year
```

```

      PaperlessBilling      PaymentMethod  MonthlyCharges  TotalCharges  \
0           Yes      Electronic check          29.85          29.85
1           No      Mailed check          56.95         1889.50
2           Yes      Mailed check          53.85          108.15
3           No      Bank transfer (automatic)      42.30         1840.75
4           Yes      Electronic check          70.70         151.65
...
7038        Yes      Mailed check          84.80         1990.50
7039        Yes      Credit card (automatic)     103.20         7362.90
7040        Yes      Electronic check          29.60          346.45
7041        Yes      Mailed check          74.40          306.60
7042        Yes      Bank transfer (automatic)     105.65         6844.50
```

```

      Churn
0      No
1      No
2      Yes
3      No
4      Yes
...
7038    No
7039    No
7040    No
7041    Yes
7042    No
```

```
[7043 rows x 21 columns]>
```

```
In [11]: df.isnull().sum().sum()
```

```
Out[11]: np.int64(0)
```

```
In [13]: df.describe()
```

Out[13]:

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges
count	7043.000000	7043.000000	7043.000000	7043.000000
mean	0.162147	32.371149	64.761692	2279.734304
std	0.368612	24.559481	30.090047	2266.794470
min	0.000000	0.000000	18.250000	0.000000
25%	0.000000	9.000000	35.500000	398.550000
50%	0.000000	29.000000	70.350000	1394.550000
75%	0.000000	55.000000	89.850000	3786.600000
max	1.000000	72.000000	118.750000	8684.800000

In [15]:

df["customerID"].duplicated().sum()

Out[15]:

np.int64(0)

In [17]:

```
def conv(value):
    if value ==1:
        return "yes"
    else:
        return "no"
df['SeniorCitizen'] = df["SeniorCitizen"].apply(conv)
```

In [18]:

df.head()

Out[18]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	..
0	7590-VHVEG	Female	no	Yes	No	1	No	No phone service	DSL	No	..
1	5575-GNVDE	Male	no	No	No	34	Yes	No	DSL	Yes	..
2	3668-QPYBK	Male	no	No	No	2	Yes	No	DSL	Yes	..
3	7795-CFOCW	Male	no	No	No	45	No	No phone service	DSL	Yes	..
4	9237-HQITU	Female	no	No	No	2	Yes	No	Fiber optic	No	..

5 rows × 21 columns

In [19]:

df.head(20)

Out[19]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity
0	7590-VHVEG	Female	no	Yes	No	1	No	No phone service	DSL	No
1	5575-GNVDE	Male	no	No	No	34	Yes	No	DSL	Yes
2	3668-QPYBK	Male	no	No	No	2	Yes	No	DSL	Yes
3	7795-CFOCW	Male	no	No	No	45	No	No phone service	DSL	Yes
4	9237-HQITU	Female	no	No	No	2	Yes	No	Fiber optic	No
5	9305-CDSKC	Female	no	No	No	8	Yes	Yes	Fiber optic	No
6	1452-KIOVK	Male	no	No	Yes	22	Yes	Yes	Fiber optic	No
7	6713-OKOMC	Female	no	No	No	10	No	No phone service	DSL	Yes
8	7892-POOKP	Female	no	Yes	No	28	Yes	Yes	Fiber optic	No
9	6388-TABGU	Male	no	No	Yes	62	Yes	No	DSL	Yes
10	9763-GRSKD	Male	no	Yes	Yes	13	Yes	No	DSL	Yes
11	7469-LKBCI	Male	no	No	No	16	Yes	No	No	No internet service
12	8091-TTVAX	Male	no	Yes	No	58	Yes	Yes	Fiber optic	No
13	0280-XJGEX	Male	no	No	No	49	Yes	Yes	Fiber optic	No
14	5129-JLPIS	Male	no	No	No	25	Yes	No	Fiber optic	Yes
15	3655-SNQYZ	Female	no	Yes	Yes	69	Yes	Yes	Fiber optic	Yes
16	8191-XWSZG	Female	no	No	No	52	Yes	No	No	No internet service
17	9959-WOFKT	Male	no	No	Yes	71	Yes	Yes	Fiber optic	Yes
18	4190-MFLUW	Female	no	Yes	Yes	10	Yes	No	DSL	No
19	4183-MYFRB	Female	no	No	No	21	Yes	No	Fiber optic	No

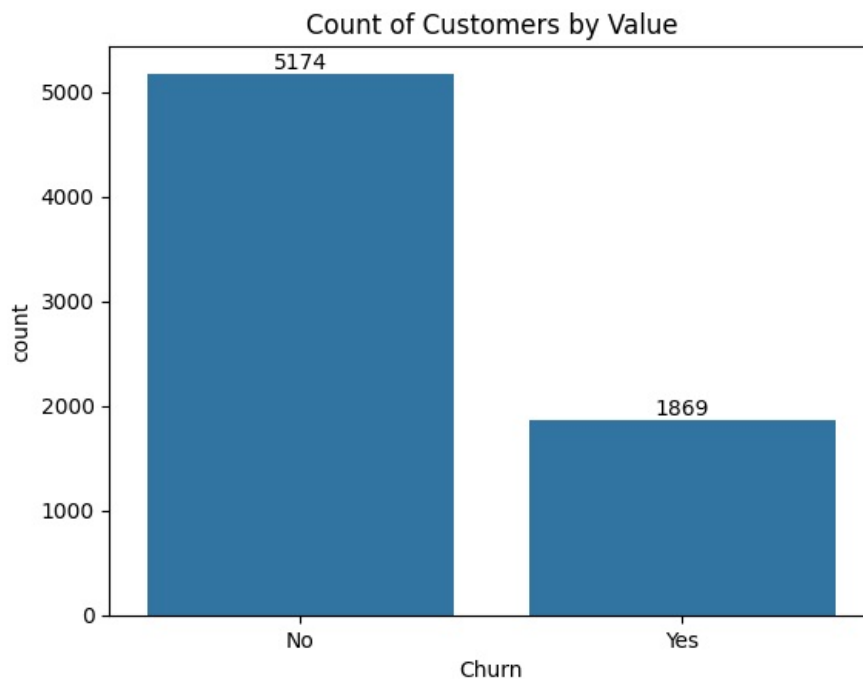
20 rows × 21 columns



In [20]: *#converted 0 and 1 values of senior citizen to yes/no to make it easier to understand*

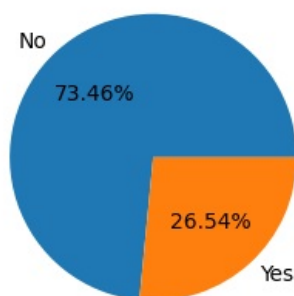
```
ax = sns.countplot(x = 'Churn', data = df)
ax.bar_label(ax.containers[0])
plt.title("Count of Customers by Value")
plt.show
```

Out[29]: <function matplotlib.pyplot.show(close=None, block=None)>



```
In [32]: plt.figure(figsize = (3,4))
gb = df.groupby("Churn").agg({'Churn' : "count"})
plt.pie(gb['Churn'], labels = gb.index, autopct = "%1.2f%%")
plt.title("Percentage of Churned Customers", fontsize=10)
plt.show()
```

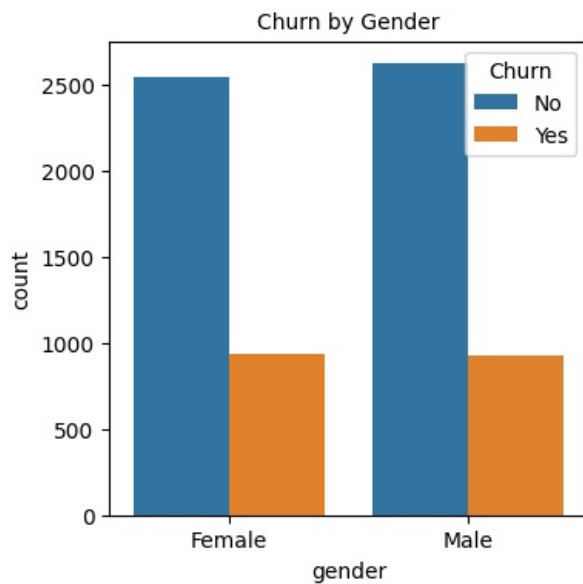
Percentage of Churned Customers



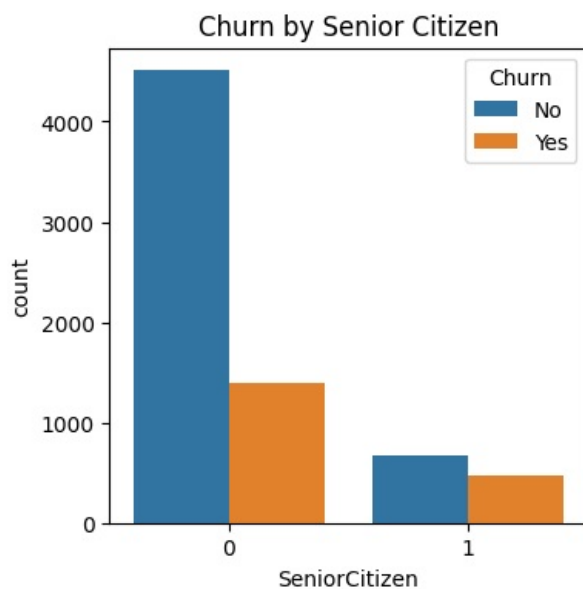
from the given pie chart we can conclude that 26.54% of our customers have churned out.

now let us explore the reason behind it.

```
In [45]: plt.figure(figsize = (4,4))
sns.countplot(x = "gender", data = df, hue = "Churn")
plt.title("Churn by Gender", fontsize=10)
plt.show()
```



```
In [6]: import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize = (4,4))
sns.countplot(x = "SeniorCitizen", data = df, hue = "Churn")
plt.title("Churn by Senior Citizen")
plt.show()
```

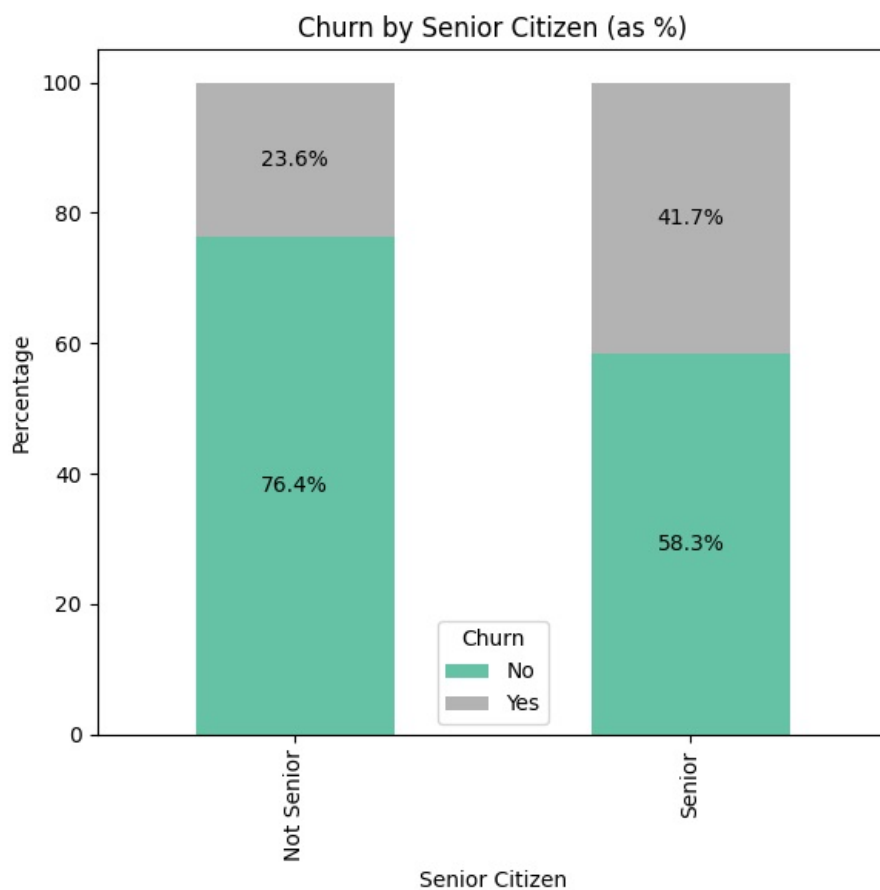


```
In [7]: crosstab = pd.crosstab(df['SeniorCitizen'], df['Churn'], normalize='index') * 100

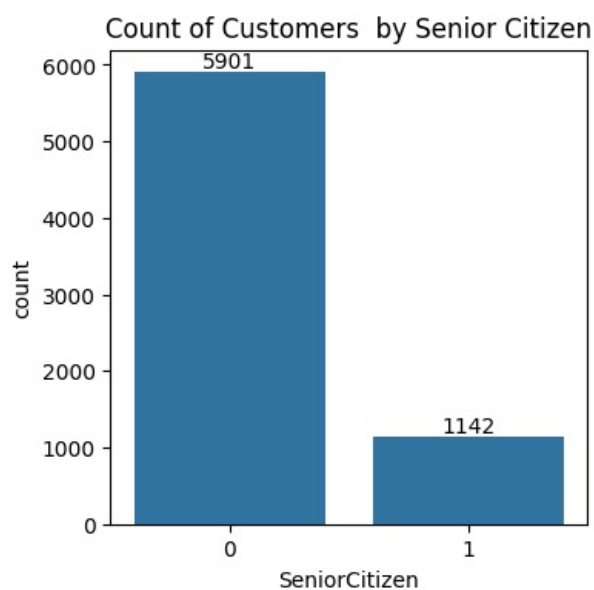
# Step 2: Plot a stacked bar chart
crosstab.plot(kind='bar', stacked=True, figsize=(6, 6), colormap='Set2')

# Step 3: Add percentage labels
for i, row in crosstab.iterrows():
    cumulative = 0
    for churn_status in crosstab.columns:
        value = row[churn_status]
        plt.text(i, cumulative + value / 2, f'{value:.1f}%', ha='center', va='center', color='black')
        cumulative += value

# Customize chart
plt.title('Churn by Senior Citizen (as %)')
plt.ylabel('Percentage')
plt.xlabel('Senior Citizen')
plt.xticks([0, 1], ['Not Senior', 'Senior'])
plt.legend(title='Churn')
plt.tight_layout()
plt.show()
```

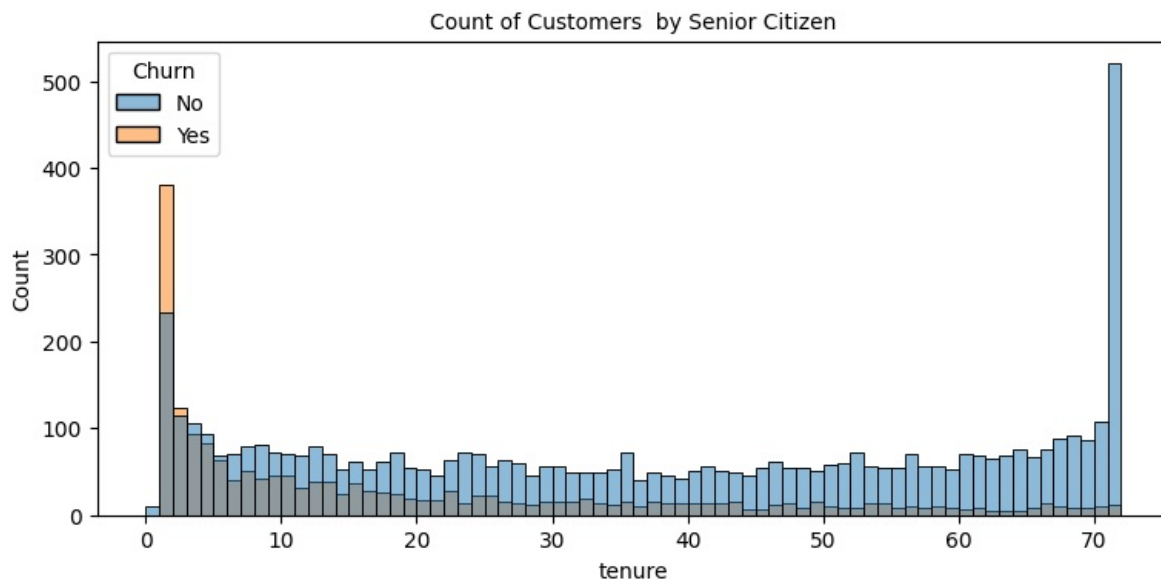


```
In [9]: plt.figure(figsize = (4,4))
ax = sns.countplot(x = "SeniorCitizen", data = df)
ax.bar_label(ax.containers[0])
plt.title("Count of Customers by Senior Citizen", fontsize=10)
plt.show()
```



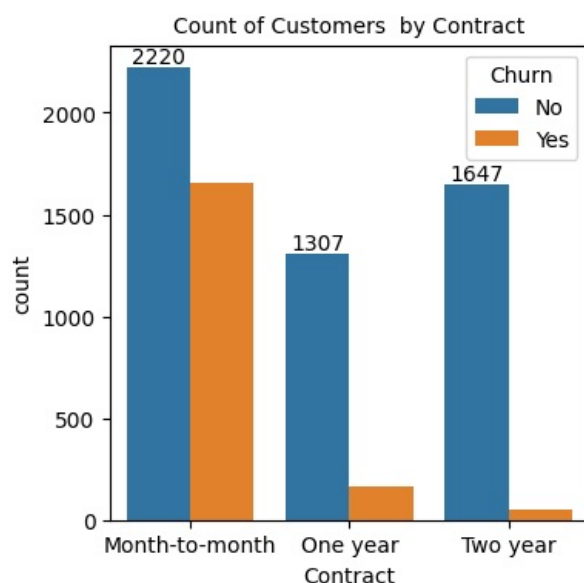
comparatively a greater percentage of people in senior citizen category have churned out

```
In [12]: plt.figure(figsize = (9,4))
sns.histplot(x = "tenure", data = df, bins = 72, hue = "Churn")
plt.title("Count of Customers by Senior Citizen", fontsize=10)
plt.show()
```



People who have used our services for a long time have stayed and people who have used our services for one or two months have churned out.

```
In [14]: plt.figure(figsize = (4,4))
ax = sns.countplot(x = "Contract", data = df, hue = "Churn" )
ax.bar_label(ax.containers[0])
plt.title("Count of Customers by Contract", fontsize=10)
plt.show()
```



People who have month to month contract are likely to churn than from those who have one or two years contract.

```
In [16]: df.columns.values
```

```
Out[16]: array(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
               'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
               'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
               'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contract',
               'PaperlessBilling', 'PaymentMethod', 'MonthlyCharges',
               'TotalCharges', 'Churn'], dtype=object)
```

```
In [20]: # Define the columns you want to plot
columns = ['PhoneService', 'MultipleLines', 'InternetService',
           'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
           'TechSupport', 'StreamingTV', 'StreamingMovies']

# Set up the subplot grid - 3 columns, automatic number of rows
n_cols = 3
```



```

n_rows = (len(columns) + n_cols - 1) // n_cols # ceiling division

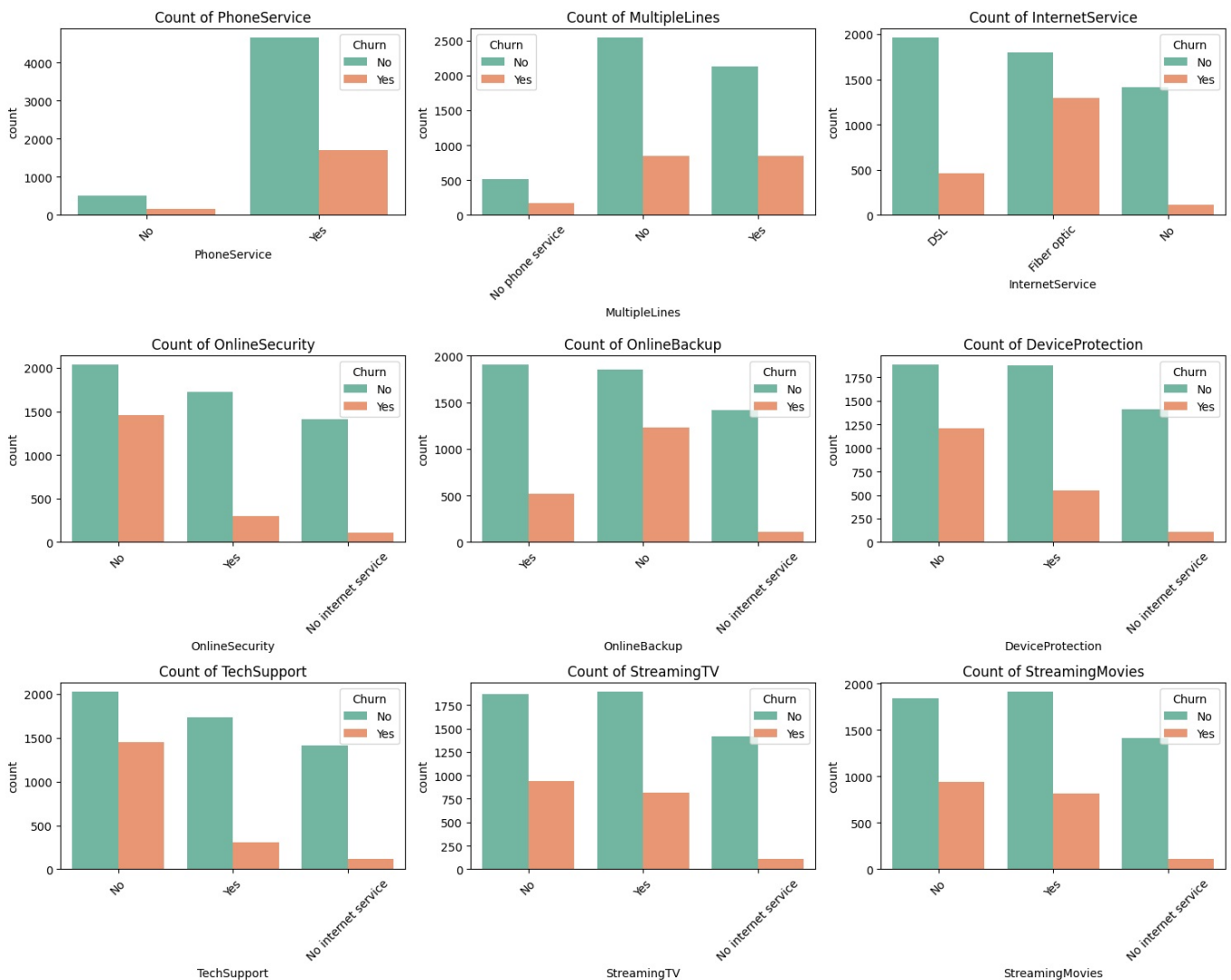
# Create figure and axes
fig, axes = plt.subplots(n_rows, n_cols, figsize=(15, n_rows * 4))
axes = axes.flatten() # Make axes iterable

# Loop over columns and create countplots
for i, column in enumerate(columns):
    sns.countplot(x=column, data=df, ax=axes[i], palette="Set2", hue = "Churn")
    axes[i].set_title(f'Count of {column}')
    axes[i].tick_params(axis='x', rotation=45) # Rotate labels for readability

# Remove unused subplots if any
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

# Adjust layout
plt.tight_layout()
plt.show()

```



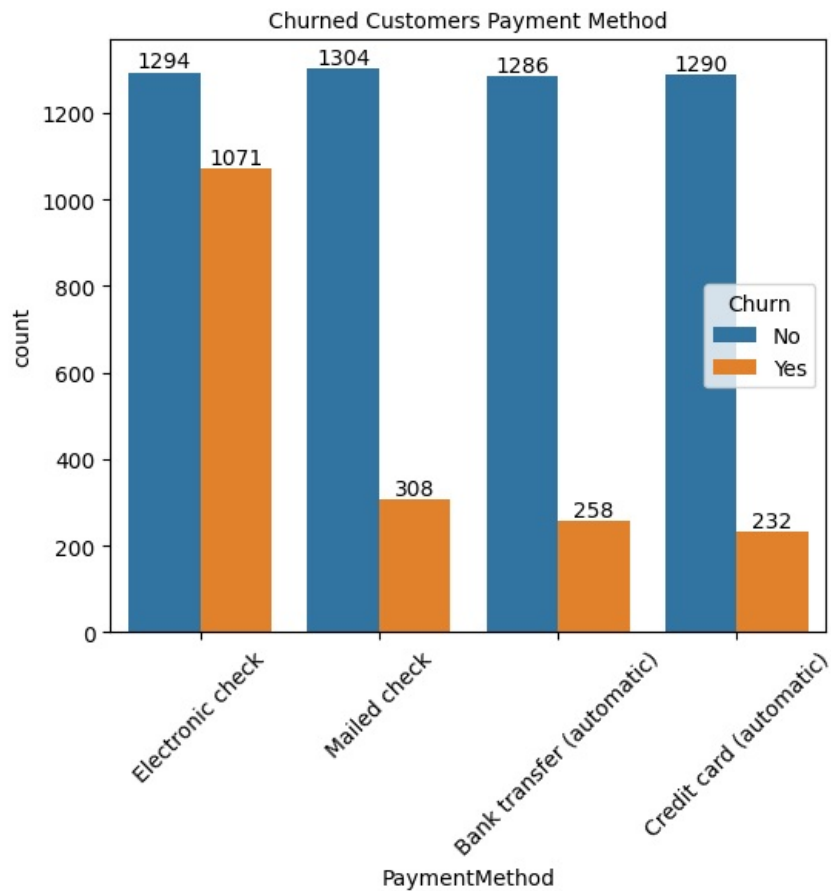
The visualizations present count plots showing customer churt (yes or no) across various services feature such as phone service, multiple lines,

internet service (particularly DSL), and Online security enabled. For services like online backup, technical support and streaming TV churn rates

are noticeably higher when these services are not used or are unavailable. The charts emphasiz the relationship between service usage and customer

churn

```
In [25]: plt.figure(figsize = (6,5))
ax = sns.countplot(x = "PaymentMethod", data = df, hue = "Churn")
ax.bar_label(ax.containers[0])
ax.bar_label(ax.containers[1])
plt.title("Churned Customers Payment Method", fontsize=10)
plt.xticks(rotation = 45)
plt.show()
```



Customer is likely to churn when he is using electronic check as a payment method